

KNN

loss function

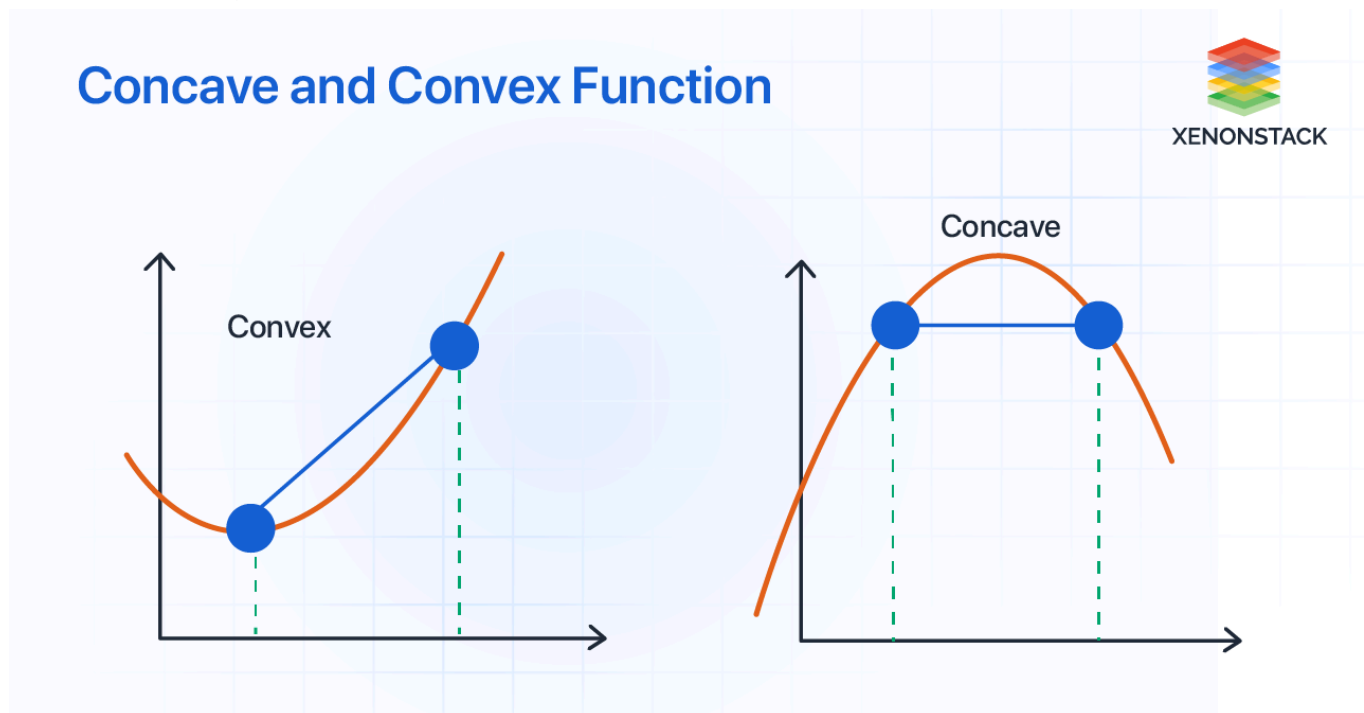
regularization form: lasso (sparse, 变成0) ridge (密集的) 不考虑, 考试的时候, 这里默认是 ridge

K(这里的是训练集分成K份, 然后K-1份训练, 1份测试之类的) fold 分割 数据集

KNN实际上需要在不同的模型上做, 第一次是为了选择超参数, 第二次是为了选择模型, 有点 old school了

LR

为什么这么设计, loss使用MSE?



Learning the linear model $\rightarrow$ learning the β 's

Estimation with Least squares:

Use least square loss: $loss(y_i, f(x_i)) = (y_i - f(x_i))^2$

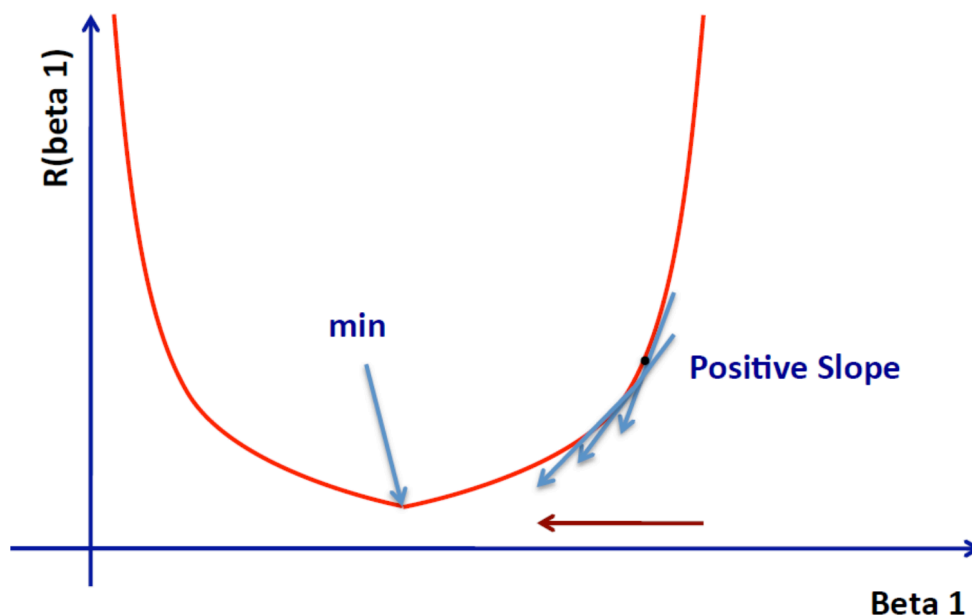
We want to minimize the loss over all examples, that is minimize the *risk or cost function* R .

$$R = \frac{1}{2n} \sum_{i=1}^n (y_i - f(x_i))^2$$

为什么这里需要除2? 因为我们要求导, 这里可以把2消掉, different design in IME

LR: Gradient descent 使用Newton's way, 梯度在二维会退化成斜率, 梯度默认是上升最快的方向 (正的), 所以我们需要使用梯度下降得到最优解

Gradient descent



Gradient descent

Gradient Descent is an optimization method.

Repeat until convergence:

Update **simultaneously** all β_j for ($j = 0$ and $j = 1$)

$$\beta_0 := \beta_0 - \alpha \frac{\partial}{\partial \beta_0} R(\beta_0, \beta_1)$$

$$\beta_1 := \beta_1 - \alpha \frac{\partial}{\partial \beta_1} R(\beta_0, \beta_1)$$

α is a learning rate.

写不写成向量都一样的, 只是向量复杂一点。

如何确保我的梯度最终收敛：

使用早停机制

分类：SVM，逻辑回归（sigmoid 函数（这是预测的函数），因为可以把R映射到一个概率区间里面，而且它的导数就是和自己相关（ $\sigma'(x)=\sigma(x)(1-\sigma(x))$ ），使用交叉熵函数（和你的预测有关系））。

In general:

$$f(x) = g\left(\sum_{j=1}^d \beta_j x_j\right)$$

How to find the β 's?

$$R(\beta) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (f(x) - y)^2$$

$$Loss = \frac{1}{2} (f(x) - y)^2$$

Logistic Regression

New Convex function: $Cost(f(x), y) = \begin{cases} -\log(f(x)) & \text{if } y = 1 \\ -\log(1 - f(x)) & \text{if } y = 0 \end{cases}$

1. If $y = 1$ if the prediction $f(x) = 1$ then cost = 0
2. If $y = 1$ if the prediction $f(x) = 0$ then cost $\rightarrow \infty$
3. If $y = 0$ if the prediction $f(x) = 0$ then cost $\rightarrow 0$
4. If $y = 0$ if the prediction $f(x) = 1$ then cost $= \infty$

Logistic Regression

Nice convex functions!

Let's combine them in a compact function (because $y = 0$ or $y = 1$):

$$Loss(f(x), y) = -y \log f(x) - (1 - y) \log(1 - f(x))$$

$$R(\beta) = -\frac{1}{n} \left[\sum_{i=1}^n y \log f(x) + (1 - y) \log(1 - f(x)) \right]$$

NN 如果处理输入X，得到结果Y

首先，对X 加权

然后，激活函数

然后，对激活函数加权，得到结果Y

逻辑

$p \rightarrow q$

$\neg p \vee q$ disjunction

$\wedge$ conjugate

推理：

条件 (condition)，命题 (inference) / 结论 (conclusion)

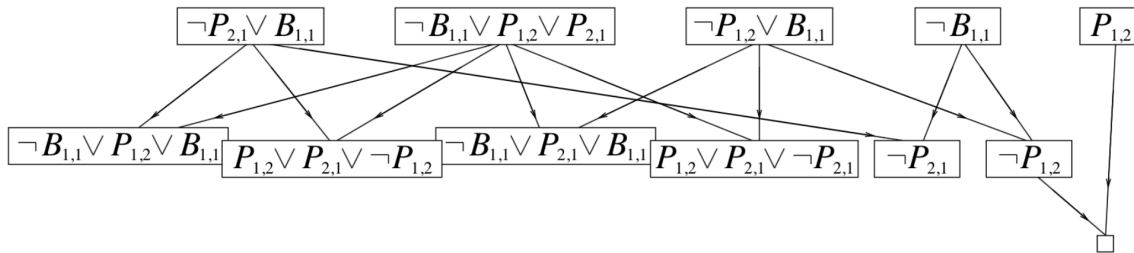
Proof by Resolution 感觉必考，老师特地讲的

Conversion to CNF 感觉必考，老师特地讲的

Resolution example

$$KB = R_4 \wedge R_2 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$$

$$\alpha = \neg P_{1,2}$$



CSP

Back tracking search 感觉必考，老师特地讲的

Backtracking search

function BACKTRACKING_SEARCH(*csp*) returns a solution, or failure
return BACKTRACK({}, *csp*)

function BACKTRACK(*assignment*, *csp*) returns a solution, or failure
if *assignment* is complete **then return** *assignment*
var = SELECT_UNASSIGNED_VARIABLE(*csp*)
for each *value* in ORDER_DOMAIN_VALUES (*var*, *assignment*, *csp*)
 if *value* is consistent with *assignment* **then**
 add {*var* = *value*} to *assignment*
 result = BACKTRACK(*assignment*, *csp*)
 if *result* ≠ failure **then return** *result*
 remove {*var* = *value*} from *assignment*
return failure

Backtracking w/inference

```
function BACKTRACKING-SEARCH(csp) returns a solution or failure
  return BACKTRACK(csp, { })

function BACKTRACK(csp, assignment) returns a solution or failure
  if assignment is complete then return assignment
  var  $\leftarrow$  SELECT-UNASSIGNED-VARIABLE(csp, assignment)
  for each value in ORDER-DOMAIN-VALUES(csp, var, assignment) do
    if value is consistent with assignment then
      add { var = value } to assignment
      inferences  $\leftarrow$  INFERENCE(csp, var, assignment)
      if inferences  $\neq$  failure then
        add inferences to csp
        result  $\leftarrow$  BACKTRACK(csp, assignment)
        if result  $\neq$  failure then return result
        remove inferences from csp
      remove { var = value } from assignment
  return failure
```

42

一步步，所小 定义域

A, B, C 1, 2, 3 A不等B, B不等C, C不等A

A=1 B 2, 3 C 2, 3

B= 2 C 3

c=3

这是一个解

如果限制 加入c不等于3

那么我们最后的记过是失败的，所以需要回溯

Arc consistency 感觉必考，老师特地讲的，最后强调的

没有代码，全是大题